

Trabant: A Serverless Architecture for Multi-Tenant Orbital Edge Computing

Tobias Pfandzelter

Technische Universität Berlin
Berlin, Germany
tp@3s.tu-berlin.de

Nikita Bauer

Technische Universität Berlin
Berlin, Germany
nba@3s.tu-berlin.de

Alexander Leis

Technische Universität Berlin
Berlin, Germany
ale@3s.tu-berlin.de

Corentin Perdrizet

Bordeaux INP
ENSEIRB-MATMECA
Bordeaux, France
corentin.perdrizet@bordeaux-
inp.fr

Felix Trautwein

Technische Universität Berlin
Berlin, Germany
ftr@3s.tu-berlin.de

Trever Schirmer

Technische Universität Berlin
Berlin, Germany
ts@3s.tu-berlin.de

Osama Abboud

Huawei Technologies
Munich, Germany
osama.abboud@huawei.com

David Bermbach

Technische Universität Berlin
Berlin, Germany
db@3s.tu-berlin.de

Abstract

Orbital edge computing reduces the data transmission needs of Earth observation satellites by processing sensor data on-board, allowing near-real-time insights while minimizing downlink costs. However, current orbital edge computing architectures are inflexible, requiring custom mission planning and high upfront development costs. In this paper, we propose a novel approach: shared Earth observation satellites that are operated by a central provider but used by multiple tenants. Each tenant can execute their own logic on-board the satellite to filter, prioritize, and analyze sensor data.

We introduce Trabant, a serverless architecture for shared satellite platforms, leveraging the Function-as-a-Service (FaaS) paradigm and time-shifted computing. This architecture abstracts operational complexities, enabling dynamic scheduling under satellite resource constraints, reducing deployment overhead, and aligning event-driven satellite observations with intermittent computation. We present the design of Trabant, demonstrate its capabilities with a proof-of-concept prototype, and evaluate it using real satellite computing telemetry data. Our findings suggest that Trabant can significantly reduce mission planning overheads, offering a scalable and efficient platform for diverse Earth observation missions.

1 Introduction

Orbital edge computing (OEC) can reduce the amount of data Earth observation satellites need to downlink by filtering and prioritizing sensor readings on-board the satellite, decreasing cost and energy demand while allowing insights

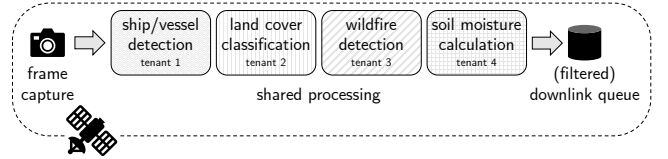


Figure 1: A shared satellite with orbital edge computing could continuously capture Earth observation data and process it using services and models from different tenants, reducing the downlink strain while still providing near-real-time insights

in near-real-time [26, 27, 32, 33, 48, 77, 78, 81, 83]. Current OEC architectures require careful mission planning, aligning sensor resolution, orbital parameters, energy harvesting equipment, algorithms, and compute capabilities for a specific use case, meaning OEC-equipped satellites are inflexible and have high upfront development costs [26, 37, 77].

Instead of these vertically integrated, purpose-built satellites, we propose *shared* Earth observation satellites, owned and operated by a satellite operator, but simultaneously used by different tenants to flexibly execute their Earth observation missions. We show a simplified version of this idea in Figure 1. Our shared satellite captures frames using its cameras and sensors, those frames and their metadata are processed on the satellite by different services, which then insert their results into a downlink queue or discard frames that are not of value to them. Essentially, clients can use their own logic on-board the satellite to filter and prioritize Earth

observation data and perform inference tasks in as soon as new data is captured. The key insights that lead us to believe that this is a viable architecture are:

- (1) That Earth observation satellites are often unused, e.g., a satellite meant to observe wildfires also flies over the oceans, and that sharing sensing capabilities can reduce overall mission costs.
- (2) That the mission planning and operating processes between satellite operators and Earth observation clients are not well aligned: While operators need months and thousands of dollars to develop a satellite and then operate it for years to amortize those costs, clients, e.g., scientists, are interested in running new live queries, continuously updating their inference and filtering logic, or deploying new measurement campaigns, all with as little lead time as possible.
- (3) That OEC already provides a flexible platform for sharing resources using the same service deployment and management approaches that are used in terrestrial computing today.

Emphasizing this last point, we present Trabant, a serverless architecture for shared Earth observation satellite platforms with OEC. We believe that following the principles of serverless computing, especially the Function-as-a-Service (FaaS) deployment paradigm, can help overcome the salient challenges of operating shared satellite platforms:

- The high level of abstraction in FaaS allows satellite operators to schedule or interrupt OEC service invocations under the unique temperature, energy, and resource constraints of an OEC satellite, e.g., allowing temporal shifting of computation to times when a satellite can harvest solar energy [65, 67].
- This level of abstraction also reduces development costs for clients, who no longer have to manage resource scheduling or other operational concerns in their OEC services [41, 44].
- Shared application runtimes reduce the size of service deployment packages, which helps reduce deployment costs given the high uplink costs for satellites. Instead of uplinking, e.g., a Python runtime per service (as would be required for containers or virtual machines), deploying a FaaS function requires only its code.
- The FaaS execution model, where client code is executed in response to events, aligns with the continuous generation of Earth observation events by satellite sensing equipment.

We elaborate the opportunities and challenges of shared satellite platforms in §3. We give an overview of the architecture of Trabant in §4 and evaluate it with a proof-of-concept prototype in §5. Specifically, we use real OEC satellite traces

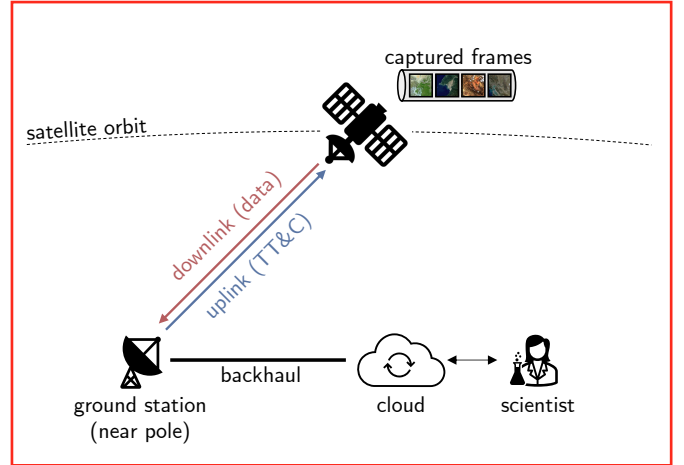


Figure 2: Traditional Earth observation satellite capture images of Earth from LEO and downlink them when passing ground stations, which are usually located near the Earth’s poles. Data is sent through the backhaul network to a cloud for processing, from which scientists can access it for analysis

and telemetry data to replicate a realistic OEC environment for this evaluation. Finally, we provide an outlook and avenues for future work in §6. We make all artifacts developed for this paper available as open-source software.¹

2 Background

We first give an overview of Earth observation satellites, orbital edge computing, and serverless edge computing.

2.1 Earth Observation Satellites

Earth observation satellites collect data on Earth’s land, oceans, or atmosphere, providing a crucial data source for remote sensing applications in climate science, agriculture, finance, or disaster monitoring. 95 % of the more than 1200 Earth observation satellites orbiting Earth today are located at altitudes below 2000 km, in low-Earth orbit (LEO) [6, 50]. This low orbit allows high-resolution data capture, limits the power required to downlink data, and is cheaper to launch into [26]. To maintain orbit, LEO satellites perform a revolution around Earth every 1 h to 2 h. Many of the Earth observation satellites active today are small, e.g., following the CubeSat standard with a standardized form factor of a 10 cm cube (1U) [23], and use commercial-off-the-shelf (COTS) equipment, keeping development costs as low as \$65 000 [2, 18, 22, 27].

We show a typical Earth observation satellite architecture in Figure 2. The satellite continuously captures frames,

¹<https://github.com/project-spencer/trabant>

i.e., images of Earth in different bands (with different central wavelengths) [26]. Satellites periodically pass ground stations that they can downlink data to [75]. Such ground stations are usually located near the Earth’s poles, as this allows the satellite to perform downlinking once per orbit despite Earth’s rotation. Depending on the satellite’s power constraints and antenna size, total downlink data size for a typical Earth observation satellite can range from **hundreds of megabytes to a few gigabytes per pass** [28, 75]. In the up-link direction, satellites can receive tracking, telemetry, and control (TT&C) data, e.g., for acknowledgements, although this link is usually narrowband and thus constrained to tens to hundreds of kilobits per second [75, 81].

A simple calculation shows that downlink rate is a key bottleneck for Earth observation: Consider a satellite capturing images at a $256 \text{ px} \times 256 \text{ px}$ frame resolution with each pixel representing $10 \text{ m} \times 10 \text{ m}$ on the ground (a relatively subpar resolution). Within a single orbit, the satellite would collect $\frac{40\,000 \text{ km}}{256 \text{ px} \times 10 \text{ m px}^{-1}} \simeq 15\,628$ individual frames. Using only red, green, and blue bands with one byte per pixel, this would yield $\sim 3.07 \text{ GB}$ of raw data to be downlinked per orbit.

2.2 Orbital Edge Computing

Orbital edge computing (OEC) is a new approach to limit the cost of data downlinking through on-board data processing [26, 27, 32, 33, 38, 43, 47, 48, 77, 78, 81, 83]. By executing filtering and inference on the satellite, unneeded data can be discarded, reducing the strain on the downlink. For example, **ESA’s Φ-SAT-1 [36, 37] mission has demonstrated on-board cloud segmentation using a machine learning (ML) model.** Beyond that, information can be extracted directly from the raw data to then downlink only an ML inference result instead of a whole frame. Such filtering and inference not only saves bandwidth, but reduces the time to generate insights from Earth observation data.

Unlike satellite flight controllers, which are critical components, OEC can use commodity COTS computing hardware, such as Raspberry Pi or NVIDIA Jetson boards [5, 8, 14, 70, 81]. The limited radiation in LEO may lead to infrequent hardware crashes as a result of single-event upsets (SEU) but has been shown to not affect performance or lifetime for typical missions [58, 81]. The main constraints for such hardware are energy availability and heat dissipation [81]. Most Earth observation satellites use a combination of solar panels and batteries [34, 61]. Depending on solar panel area and sunlight, a CubeSat can harvest on the order of 1 W to 150 W of energy at maximum [9, 34, 81]. If the satellite is eclipsed by Earth or not ideally positioned for full solar exposure, less or no energy can be harvested. During this time, the satellite may use its batteries to drive communication and computation equipment.

Given the lack of atmosphere in LEO, satellite on-board computing devices must make use of radiative cooling, which is relatively inefficient especially for smaller satellites with little surface area [81, 82]. Such passive heat dissipation using the satellite structure must also ensure that the operational temperature limits of the satellite are not exceeded, as this could permanently damage satellite components. Consequently, there is a limit on how much heat a computing chip may generate during its operation.

2.3 Serverless Edge Computing

Serverless computing is a popular cloud deployment paradigm that decouples application logic from operational concerns such as elastic scaling [41, 44]. Function-as-a-Service (FaaS) is one of the predominant serverless programming models. In FaaS, applications are composed of small, stateless functions that can be invoked in response to events, such as HTTP requests, asynchronous messages, or changes to a database. Developers only write the function code using a high-level programming language, while the underlying platform, which is managed by the cloud provider, handles resource allocation, elastic on-demand function instantiation, and scale-to-zero. This benefits not only application developers, who can outsource operational concerns and focus on business logic, but also cloud platform operators, who can allocate their available resources efficiently and dynamically pool resources between tenants [56].

At the edge, where resources are more constrained than in a cloud data center, FaaS can have similar benefits [15, 27, 62, 69, 80]. Rather than allocating fixed ‘slices’ of infrastructure for a tenant, an edge service provider can dynamically allocate the limited edge resources between functions of different tenants, creating the illusion that each tenant can always use as many resources as they require.

3 Multi-Tenant Orbital Edge Computing

Before introducing the Trabant architecture, we elaborate how we believe existing satellite mission design falls short of the benefits that OEC has, and how shared OEC satellites could address these limitations by decoupling the satellite platform from the applications and missions running on it.

Mission Cost. An obvious disadvantage of planning a separate satellite mission for each application is cost. Even with COTS equipment, parts costs for a 1U CubeSat can be on the order of $\$65\,000$ [2, 26]. Launch costs for commercial ride-share launches can be on the order of $\$2500/\text{kg}$ [76], but additional costs for shipping, installation, payload separation, and operation can increase this to $\$100\,000$ per mission [54]. In addition, planning a satellite mission, building the satellite, and operating it over a multi-year lifetime require significant human resources. These costs are prohibitive for most Earth

observation use-cases, where scientists must instead rely on public commercial Earth imagery providers with lower-resolution data that can be days or weeks old. *A shared satellite platform could distribute these missions costs between the tenants over the lifetime of the satellite.*

Mission Lead Times. Along with mission development costs, satellite missions also have long lead times, including manufacturing the hardware itself, licensing the satellite and communication frequencies, and waiting for a suitable launch opportunity. CubeSat missions regularly need 1–2 years from start to the satellite being inserted into orbit [54]. Application requirements regarding sensor resolution, downlink bandwidth, and compute resources have to be known early in this process. These mission lead times mean that any OEC software meant to provide valuable insights from orbit is already years old by the time it becomes operational, a significant time span compared to advances in software development. *Decoupling OEC software development from satellite development can reduce lead times for new applications to months or even weeks.*

Mission/Hardware/Software Alignment. When planning a satellite mission for a specific application, mission design has to be carefully balanced to manage costs. As weight is the main factor for launch costs, antenna size, solar panel area, battery capacity, OEC compute equipment, and sensors must be chosen to fit perfectly. For example, harvesting too little solar power impacts the functionality of the satellite, while harvesting too much means that some launch weight was wasted. Similarly, the compute equipment must be able to run the filtering or inference logic in the time it takes to capture a new frame, i.e., not lead to queuing, yet executing the logic too quickly implies that OEC equipment was over-provisioned. Then, the downlink bandwidth (and antenna size and power budget) must fit the expected data size, i.e., the frame resolution, frame capture frequency, and expected data minimization through the OEC software. While these constraints can of course be solved with careful mission planning, they significantly constrain the flexibility of the satellite. Updating the mission, e.g., with new filtering logic that is more accurate but requires more compute resources or leads to more total downlinked data, is not possible. As a result, the data that an Earth observation satellite generates over its lifetime can become less valuable as the mission progresses. *Running multiple applications on a shared satellite platform with OEC allows flexible allocation and reallocation of the mission to different applications.*

Development Cost for Constrained Software. OEC is a unique environment to deploy and manage application services in. OEC software must deal with varying energy availability, using excess solar power when the satellite is in sunlight

and not entirely depleting the battery when the satellite is eclipsed. At the same time, the software must refrain from sustained resource utilization, as this could increase equipment temperature beyond operational limits. Finally, while unlikely in LEO, SEU caused by radiation outside the Earth’s atmosphere may lock up the OEC computer, necessitating a hard reboot across which the software must maintain functionality. While not impossible to design for, these constraints are hardly intuitive for those providing the OEC application logic, who are experts in Earth observation or related fields rather than OEC software developers. Additionally, the effort to develop software to fit these constraints is repeated for every satellite mission, as there are no existing OEC operating systems to build on top of. *A shared satellite OEC environment could provide a software platform tailored to the satellite, abstracting these constraints for the application services running on it.*

LEO Satellites Spend Little Time Over Their Regions-of-Interest. To maintain their orbit, satellites in LEO must move at speeds in excess of 25 000 km/h, while Earth continuously rotates underneath. Over time, the satellite covers different geographical areas that may not be of interest to its mission. For example, a satellite monitoring wildfires in Australia from 520 km altitude in a typical sun-synchronous orbit (97° inclination) will spend only 2.5 % of its lifetime over Australia [25, 49]. Of course this satellite could be designed to wake up and capture frames only over a specific geographic area, with power, downlink, and compute resources budgeted accordingly, although the sensors could also be used for other missions. From a cost perspective, fixed development and operational costs remain. *Serving multiple tenants and applications means that a shared satellite can provide more value as it visits different geographic areas during each orbit.*

Environmental Impact. Finally, the environmental impacts of having hundreds or thousands of application-specific satellites in LEO are worth considering. Each satellite launch releases considerable amounts of greenhouse gases into the atmosphere and contributes to stratospheric ozone depletion [64]. During its mission, the satellite requires a portion of LEO, becoming a collision risk for other satellites [17, 45] and possibly contributing to light pollution [52]. Most LEO satellites deorbit after some time as a result of atmospheric drag, yet burning up in the atmosphere is suspected to negatively impact Earth’s ozone layer [31]. *Although a shared satellite may be larger than an application-specific one, reducing the overall number and mass of satellites in Earth orbit can reduce the environmental impact.*

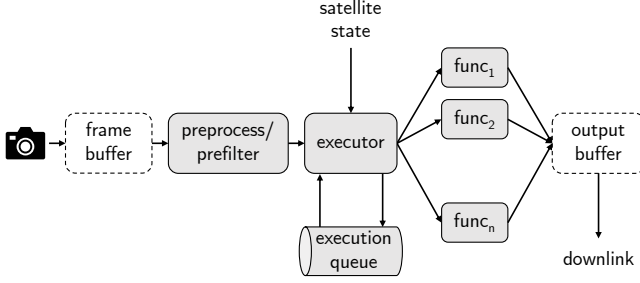


Figure 3: Trabant is a serverless architecture for on-board processing of Earth observation frames. Captured frames are preprocessed and prefiltered, with an executor either directly invoking tenants’ functions or enqueueing them when temperature or power constraints do not allow processing. The function output, which can be filtered or inferred data, is then buffered for downlinking.

4 Trabant Approach

Trabant is a serverless approach to building a shared OEC satellite software platform. Clients, such as scientists, provide filtering, inference, and selection logic for the satellite as FaaS functions. The satellite continuously captures multi-spectral frames of Earth at its given sample distance and rate. Trabant takes these frames and executes the clients’ logic against them, providing isolation for client code and ensuring that the constraints of the satellite regarding power consumption and temperature are adhered to. At the same time, it decouples the effort of managing the complex OEC execution environment from function implementation.

We provide an overview of the Trabant components in Figure 3. Captured frames first reach the *frame buffer*, the interface between the camera/sensor subsystem on the satellite and the OEC computer. Raw frame data must be adjusted for optical distortion before it can be used by an application and atmospheric turbulence or cloud cover could obscure the view of Earth. As applications expect accurate data without significant cloud cover or distortions [37, 71], Trabant includes a *preprocessing/prefiltering* step. This step, which runs on the OEC hardware, already filters out frames that do not meet a certain quality level. Note that this means that the rate at which applications receive frames for processing may be lower than the actual satellite frame capturing rate.

An *executor* invokes the different applications with the frame and its metadata, such as current time or location. The executor does not necessarily invoke each function instantly: Instead, it continuously monitors the state of the satellite, including battery charge levels and temperature. If the available energy is too low or temperature too high, the executor puts invocations (pairs of function and frame

references) into a persistent execution queue. Once sufficient energy is available and the thermal state permits, the queue can be processed. Time-shifting FaaS invocations in this manner is a well-known technique employed, e.g., in Meta’s *XFaaS* [65, 67].

Each tenant application runs as a FaaS function in its own isolated process. An implementation of Trabant may provide different language and library runtimes, e.g., for Python or a specific ML library. The results of each invocation are transferred into an output buffer that can be downlinked once the satellite is in contact with a ground station. These results could be whole frames that an application deems worthy of downlinking, cropped frames with higher value for the client, or just inference results.

While specific pricing strategies are out of scope for this paper, we envision using the same usage-based pricing strategies used in cloud computing today. This incentivizes users to limit their resource usage and allows operators to amortize cost. While the specific pricing strategies are outside the scope of this paper, we discuss avenues for future works in this field in §6.

Beyond ensuring that OEC equipment only performs processing tasks when energy is available and the satellite temperature is not close to its operational limits, note that Trabant keeps no state other than the persistent execution queue and output buffer. As a result, there is no significant impact on applications during an unexpected OEC reset, e.g., caused by SEU or when too much energy is required for communication or satellite attitude control. Each independent step, including the application functions, is free of side effects and can thus be repeated if necessary. Note that this still assumes that the actual hard drive, e.g., flash storage, is sufficiently resilient to SEU, which is an orthogonal challenge that is also present without Trabant [24, 42].

Of course, simply offering the satellite OEC resources as a platform for tenants does not mean that there are no energy, thermal, computational, or downlink constraints for the entire OEC software system. These constraints still limit the overall computational capabilities of the satellite and allocating more client services than the satellite can handle will inevitably lead to backpressure. We believe that the FaaS model significantly simplifies reasoning about the resource consumption of the services running on the shared satellite. Specifically, we argue for modelling the energy consumption of each function independently by executing it with a sufficiently large (realistic) set of input frames on compute hardware identical to that of the OEC satellite and measuring the average energy required for each invocation. The high level of abstraction that the function is programmed for makes this trivial. Further, this should also give insight to the compression ratio of the service, i.e., the number of bytes it outputs for downlinking compared to the number

of input bytes (frame count and size). These two variables must be considered for each function addition or replacement. Along with some constants specific to the satellite (which can be quantified during satellite development), the following constraint must hold when considering a change to the functions $f_1, f_2, \dots, f_n$ on the OEC satellite:

$$P_{\text{generated}} \geq P_{\text{compute}} + P_{\text{comm}} \quad (1)$$

, i.e., the power harvested for compute and communication must be greater than that spent, where:

$$P_{\text{compute}} = P_{\text{base}} + E_{\text{pre}} \times R_{\text{frame}} + \sum_{i=1}^n E_{f_i} \times R_{\text{frame}} \times (1 - R_{\text{filter}}) \quad (2)$$

Here, E_{base} is the base power draw of the OEC equipment and Trabant, E_{pre} is the energy required to preprocess a frame, R_{frame} is the frame capture rate, R_{filter} is the rate at which frames are discarded by the preprocessing step, and E_{f_i} is the energy required to execute one invocation of f_i . Note that this assumes power consumption to grow linearly with compute utilization. This is a simplifying assumption but serves as a starting point and are sufficient to investigate the Trabant approach.

Communication power budget can be calculated as:

$$P_{\text{comm}} = E_{\text{sendbyte}} \times \sum_{i=1}^n C_{f_i} \times R_{\text{frame}} \times (1 - R_{\text{filter}}) \times S_{\text{frame}} \quad (3)$$

Here, E_{sendbyte} is the energy required to downlink a single byte, C_{f_i} is the compression ratio of f_i , and S_{frame} is the size of a frame in bytes.

Next to energy budget, we must also ensure that the downlink budget is not exceeded:

$$B_{\text{downlink}} \geq \sum_{i=1}^n C_{f_i} \times R_{\text{frame}} \times (1 - R_{\text{filter}}) \times S_{\text{frame}} \quad (4)$$

, where B_{downlink} is the budget for data that can be queued for downlinking, which can be calculated by dividing data rate during ground station contact by contact rate.

Finally, we must ensure that frames can be processed as quickly as they are captured. Specifically, it must hold:

$$R_{\text{frame}}^{-1} \geq T_{\text{pre}} + \sum_{i=1}^n T_{f_i} \times (1 - R_{\text{filter}}) \quad (5)$$

, where T_{pre} is the time taken to preprocess a frame and T_{f_i} is the time taken to process one frame with f_i . Note that this ignores possible benefits from parallelization of processing on multicore architectures and thus serves as an upper bound.

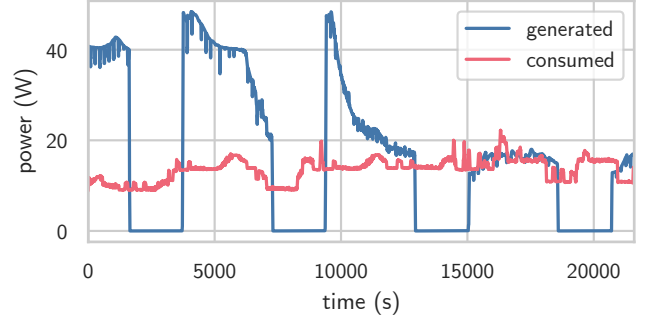


Figure 4: During our 6-hour trace, BUPT-1 consumes a mean 13.51 W of power, while its solar array generates between 0.00 W (darkness) and 48.59 W (maximum at sunlight) of power. Power output is also unstable during sunlit periods, with more power generated at the beginning than at the end, a result of the satellite’s angle relative to the sun.

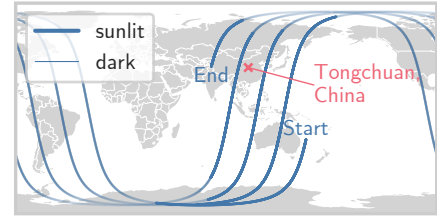


Figure 5: Our 6-hour trace from May 1st, 2023, encompasses four orbits, with the satellite coming into contact with the ground station in Tongchuan toward the end of its third orbit

5 Evaluation

To evaluate Trabant empirically, we replicate an OEC satellite in a local testbed, deploy a prototype implementation of Trabant on this testbed, and use satellite traces and ML workloads on this system.

5.1 Evaluation Environment

5.1.1 Scenario. We base our evaluation on architecture and traces of the BUPT-1 satellite [81]. BUPT-1 is a 12U Cube-Sat research satellite hosting a Raspberry Pi 4B compute board for general-purpose computation. Traces for BUPT-1 include generated solar power over time, energy spent on satellite operations, communication and computation, and temperature measurements, all at one-second granularity. BUPT-1 orbits Earth at altitudes between 487 km and 494 km with an inclination of 97.3°. BUPT-1 can harvest up to 40 W of solar power from two solar arrays and has rechargeable

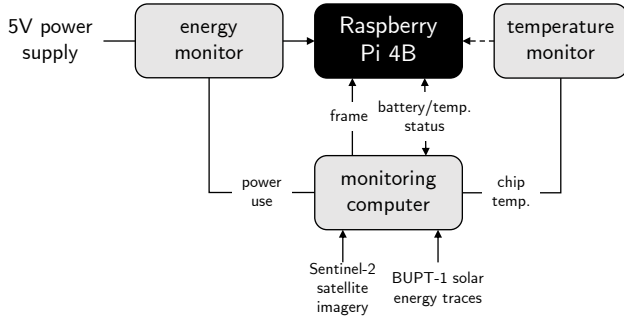


Figure 6: The local testbed architecture we use to evaluate Trabant contains a Raspberry Pi 4B hosting our Trabant prototype and workloads. A monitoring computer reads energy and temperature of the Raspberry Pi, models battery levels, and sends input frames.

lithium batteries to store energy. The batteries have a total capacity of 115 W h but must not be discharged further than 30 %, which would shorten their lifetime. As the batteries are overprovisioned and designed to power four computers instead of just a single Raspberry Pi, we use a more constrained 57.5 W h battery capacity in our experiments (one fourth of the original capacity). To communicate with the satellite, a ground station in Tongchuan, China is available with a 1 Mbps uplink and 100 Mbps downlink. We assume a 20 % protocol and error-correction overhead for each of these links. We use energy and trajectory traces of BUPT-1 for a 6-hour period on May 1st, 2023. We show the generated and consumed (by the satellite) power of our 6-hour section of the trace in Figure 4 and the satellite trajectory in Figure 5.

We base our evaluation on BUPT-1 as it is the only satellite equipped with COTS computing equipment that has detailed traces openly available with hardware that can be realistically replicated in a lab environment. Of course, there are more powerful Earth observation satellites in terms of energy harvesting efficiency, compute resources, downlink bandwidth, or ground station pass frequency. Nevertheless, as we discuss in §6, increasing any of these parameters does not obviate the need to adhere to any constraints when scheduling compute services on an Earth observation satellite, it simply changes them.

5.1.2 Testbed. For our evaluation, we replicate the BUPT-1 environment in a local testbed. As shown in Figure 6, we use a Raspberry Pi 4B as our compute module. A monitoring computer connected to this Raspberry Pi over Ethernet emulates the satellite environment by monitoring its chip temperature and overall energy consumption. Using traces from BUPT-1 solar energy harvesting with one-second granularity and measured energy consumption, we model the battery charge levels over time. This information, along with

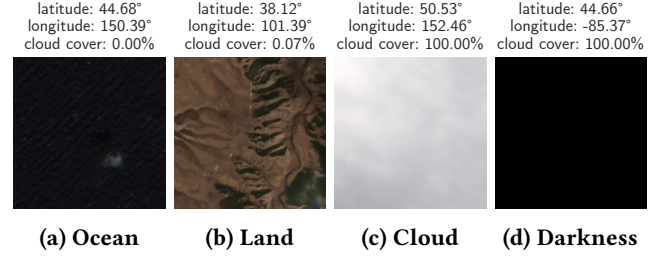


Figure 7: Example frames of our workload, each covering 2560 m × 2560 m at 10 m/px. Sunlit frames (Figures 7a to 7c) are from Sentinel-2 satellites [3, 29], dark frames are from VIIRS imagery [7, 21].

the current external chip temperature, is provided to our Trabant prototype over an HTTP API (in lieu of a real satellite interface that would provide this information). Note that in their evaluation of the BUPT-1 satellite, Xing et al. [81] have shown that a ground-based replica of the on-board Raspberry Pi 4B has similar energy and temperature performance, leading us to believe that our ground-based experiments are representative of performance in LEO.

5.1.3 Prototype. Our proof-of-concept prototype of Trabant and our monitoring are implemented in Go, loosely based on the *tinyFaaS* edge serverless platform [57]. Frames are sent to our prototype as binary data over HTTP, where frames that are not sunlit are filtered out. Next, our frames are preprocessed with an implementation of the Braaten-Cohen-Yang pixel-level cloud detection algorithm [19], providing an estimate of frame cloud cover. Frames with a cloud cover larger than 30 % are not processed further, others are enqueued for processing by our functions. Our executor dequeues function calls if it is idle, meaning the battery is not discharged more than 30 % and the external chip temperature is below 50 °C. We use this low temperature limit to approximate the difficulty of heat dissipation in space. Each function handler is a Docker container using a Python3 runtime and exposing an HTTP endpoint for invocations. Our Python3 runtime has common ML dependencies such as the TensorFlow Lite runtime pre-installed. For every frame, each function receives frame metadata, such as latitude, longitude, and cloud cover, and a path to the frame data on a shared logical volume. The function handler then ensures that any returned data is persisted as output data for downlinking. We emulate downlinking by reading from the output buffer at the downlink rate when the satellite is in contact with the ground station, modelling antenna power consumption accordingly.

5.1.4 Workload. Our monitoring computer provides a constant stream of “captured frames” as an input workload. This

Table 1: Input Parameters for Our Scenario and Prototype

Variable	Value		Source
$P_{\text{generated}}$	2950	mW	Satellite Trace
P_{base}	1518	mW	Measured
E_{sendbyte}	2	$\mu\text{J/B}$	Satellite Info
E_{pre}	0.01	J	Measured
R_{frame}	2.5	Hz	Satellite Trace
R_{filter}	77.0	%	Frame Trace
S_{frame}	$256 \times 256 \times 13$	B	Frame Trace
B_{downlink}	600	kbps	Satellite Trace
T_{pre}	0.038	s	Measured

trace is based on *Sentinel-2* [3, 29] and *Visible Infrared Imaging Radiometer Suite* (VIIRS) [7, 21] satellite imagery with the 13 Sentinel-2 bands. At a ground resolution of 10 m/px and frame size of $256 \text{ px} \times 256 \text{ px}$, the satellite would capture a new frame every 400 ms. For each of those frame areas, we first calculate if the image would be in darkness or sunlight. For dark images (59.2 %), we use night data captured by the VIIRS instrument. For sunlit frames (40.8 %), we download the most recent image of the frame location before May 1st, 2023, or use a random ocean frame (Sentinel-2 does not provide acquisitions of most of the world’s ocean area). We show example frames in Figure 7. Preliminary analysis of our collected sunlit frames with the *s2cloudless* [16] ML cloud detector shows a total cloud fraction of 36.3 %, which is smaller than expected in reality (mean ~ 67 % world cloud cover [46]), but ensures that our workload size is not favored towards our experiments. In total, 44.4 % of our frames have a cloud cover larger than 30 %.

5.1.5 Functions. We use five ML functions as our on-board workloads, each implemented using Python and TensorFlow. Each model performs ML inference on single satellite frames using the TensorFlow Lite runtime. Note that while all ML models we use are trained on real data and able to perform their respective tasks with acceptable accuracy, the goal of our functions is to present a realistic workload rather than the best possible model performance. As such, our models are also designed to have small footprints (model weights on the order of 0.5 MB to 2 MB) in order to run on constrained hardware.

Methane leak preprocessing. The goal of the methane function is to classify satellite images to filter out those that do not show industrial areas. The classification is performed by a convolutional neural network (CNN) trained on the *EuroSat* [40] dataset and based on the RGB bands of the image. If an image has been classified as showing an industrial area,

the RGB bands along with two short wave infrared bands (helpful for identifying methane leaks [74]) are stored for downlinking.

Soil moisture preprocessing. The moisture function selects images that are predicted to show different agricultural areas, such as vineyards, based on a CNN trained on the *BigEarthNet* [73] dataset. This CNN uses twelve input bands to perform multilabel classification, i.e., an image can show more than one of the classes. If the frame is relevant, the RGB bands along with two visible and near infrared bands and one short wave infrared band are stored for downlinking, as these bands can be used to infer soil moisture in subsequent processing [39].

Segmentation in China. The segment function only processes frames that show China based on a simple latitude/longitude bounding box. If a frame falls within the bounding box, the function uses a CNN model based on the *U-Net* architecture [63] to perform pixel-level segmentation, identifying features such as buildings or roads. The model is trained on a dataset of segmented satellite images of Dubai [1]. For every processed frame, the function stores the RGB bands and pixel mask for downlinking.

Vessel recognition. The vessel function contains a CNN model trained on the *MASATI* [13, 66] dataset and predicts whether a processed frame contains a boat or not. Before performing inference, the function filters out frames that have a cloud cover larger than 10 %, which could lead to inaccurate inference results. If the model decides that the frame may contain a boat with more than 80 % probability, the RGB bands of the frame are saved for downlinking.

Wildfire risk detection. The wildfire function uses a CNN model trained on the *Wildfire Prediction Dataset* [11] to identify whether the area in a frame is susceptible to wildfires or not. If the probability for a frame is higher than 60 %, all bands of the image are stored for downlinking, enabling further analysis on the ground.

5.2 Analysis

Before deploying our prototype and workload on our testbed, we explore if and when the constraints proposed in §4 hold for our workload. Table 1 shows the variables of our particular satellite environment and implementation, calculated from the satellite and image traces or measured in our prototype. In the full BUPT-1 energy monitoring trace, there is a mean 2950 mW available power, the difference between a mean 15.59 W solar power and 12.64 W power draw for satellite upkeep (without payload or data transfer).

Further, from a 80 Mbps downlink rate and 20 W antenna power, we arrive at $2 \mu\text{J/B}$ energy required to downlink one

Table 2: Measured Parameters for Our Workload Functions Across Three Repetitions (Experiments Use Values of Repeat 2)

Function	E_f			T_f			C_f		
	1	2	3	1	2	3	1	2	3
methane	0.038J	0.041J	0.036J	0.019 s	0.018 s	0.018 s	0.049	0.051	0.045
moisture	0.107J	0.092J	0.092J	0.050 s	0.050 s	0.049 s	0.029	0.041	0.018
segment	0.663J	0.755J	0.826J	0.429 s	0.494 s	0.528 s	0.043	0.047	0.051
vessel	1.075J	1.053J	1.017J	0.562 s	0.585 s	0.566 s	0.024	0.026	0.021
wildfire	0.709J	0.667J	0.719J	0.356 s	0.353 s	0.356 s	0.011	0.026	0.022
no-op	0.007J	0.007J	0.010J	0.004 s	0.004 s	0.005 s	0.000	0.000	0.000

byte. From the satellite’s trajectory over time, we gather that it is in contact with its ground station for only 0.75 % of its time in orbit, making the downlink budget $B_{\text{downlink}} = 600$ kbps. The mean filter rate R_{filter} is a result of the 40.8 % sunlit frames of which 44.4 % have an estimated cloud cover of less than 30 %. We measure P_{base} , E_{pre} , and T_{pre} on our testbed using a no-op function and our energy monitor.

We use a similar approach to quantify function parameters shown in Table 2. On our testbed, we deploy each function in isolation and send 500 randomly selected frames (only sunlit frames with cloud cover lower than 30 %) at a reduced rate (every 2 s), measuring energy required to process each frame (measured power for the duration of the call minus the base power consumption), time to process each call, and compression ratio of each function (output data size divided by input data size). For reference, we also include data on a no-op function. Note that this approach does not require any knowledge about the inner workings of a function: As we only measure the mean time or energy use across many individual calls, skewed distributions in execution behavior can be accounted for. For example, despite the segment function ignoring frames outside of China (completion within 4 ms) and executing complex inference for the rest (completion time within ~ 3 s), we receive a stable mean result across three independent repetitions of the measurement. We use the results of the second repetition of each measurement for the remainder of this experiment.

Based on the constraints we set in §4, we can now calculate which functions we may deploy as our workload. We find that both our power budget $P_{\text{compute}} = 3.02 \text{ W} > P_{\text{generated}}$ and downlink budget $746.01 \text{ kbps} > B_{\text{downlink}}$ are exceeded when considering deploying all five functions. We select the methane, moisture, vessel, and wildfire functions, which have a $P_{\text{compute}} = 2.59 \text{ W}$, $P_{\text{comm}} = 0.14 \text{ W}$, and total processing time of $0.266 \text{ s} < R_{\text{frame}}^{-1} = 0.4 \text{ s}$, and downlink 564.34 kbps.

5.3 End-to-End Demonstration

We first take a high-level look at the performance of the Trabant approach using an end-to-end demonstration of our prototype with the full 6-hour satellite trace. We use the methane, moisture, vessel, and wildfire functions and start with 60 % battery capacity and an empty queue.

We show the results of this experiment in Figure 8. Figure 8a shows the power generation and consumption of our satellite, communication, and computation. We observe that computing is mostly performed during periods of net positive satellite power, as those are sunlit periods that yield both high solar panel output and sunlit frames for processing. The satellite comes into contact with its ground station 15 189 s into the experiment, when it starts downlinking data for 270 s, with communication power increasing to 20 W accordingly. The battery level shown in Figure 8c are mostly affected by satellite power, although we see a small dip when communication starts. As there is little solar power during the last sunlit period in our trace, the battery charge drops below 70 % towards the end of our experiment. We see this reflected in queue size and processing periods in Figure 8b, where Trabant stops processing new frames (instead queuing them for later), despite a positive power output of the solar panels. Here, we also see the impact of external chip temperature, as shown in Figure 8d. Temperature increases as Trabant starts processing function calls for incoming frames. Although there is sufficient energy available for the majority of our experiment, Trabant dequeues function calls only until the chip temperature limit of 50 °C is reached. Processing these calls increases chip temperature further to a maximum 51.5 °C, yet Trabant waits until the compute device has cooled enough to dequeue more invocations.

5.4 Exceeding Constraints

To further explore how Trabant, we perform additional experiments using a subset of our trace starting at 4000 s, shortly before the start of the first full sunlit period. We compare the same setup (‘default’) as in the full experiment with two

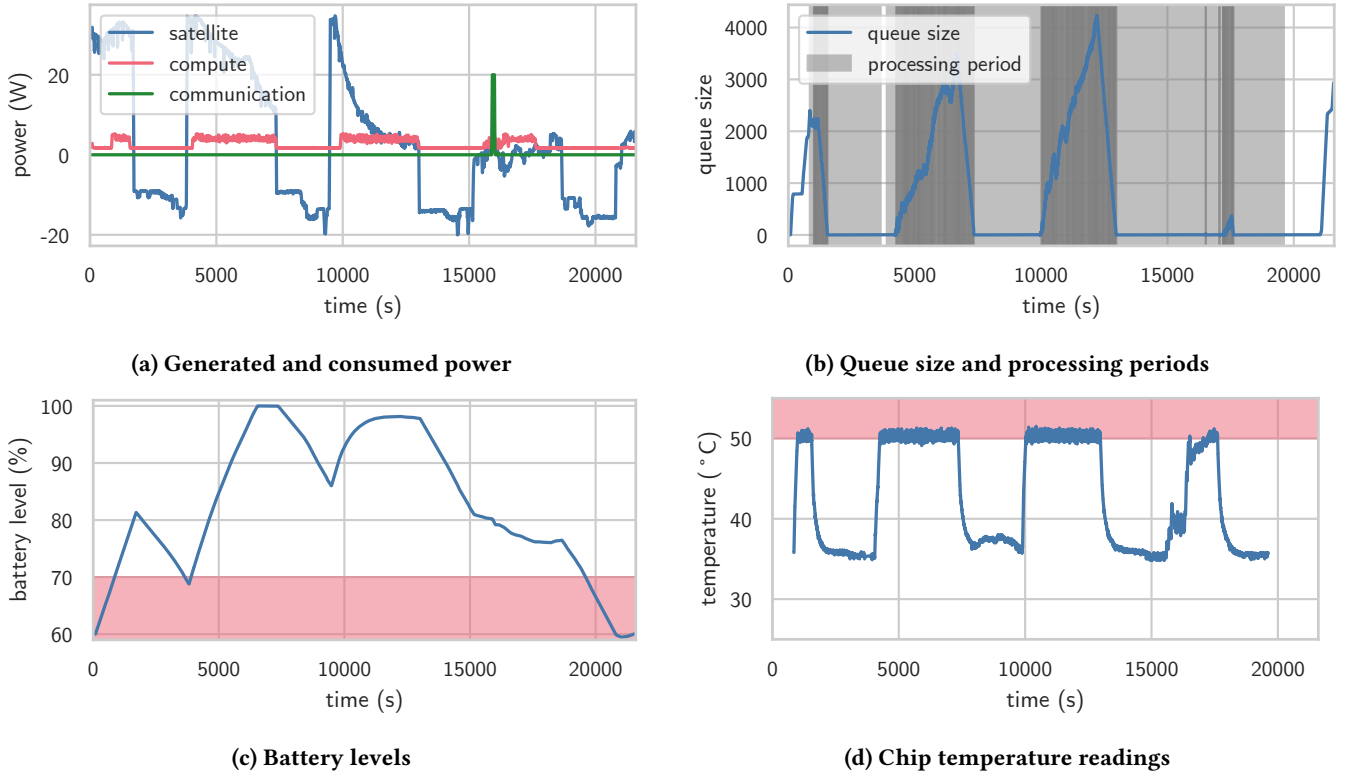


Figure 8: Results of the 6-hour experiment show how Trabant time-shifts frame processing under energy and temperature constraints

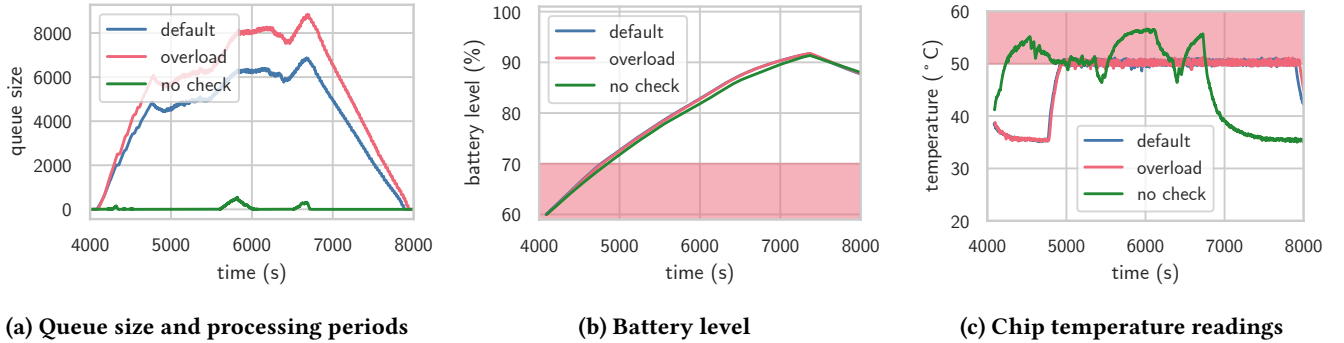


Figure 9: Results for a 4000 s experiment with additional comparison configurations. ‘overload’ adds the segment function, while ‘no check’ disables the temperature and battery level checks.

additional configurations: In the ‘overload’ configuration we add the segment function to our workload. In the ‘no-check’ configuration, we disable compute time-shifting by removing the temperature and battery level checks. Further, we start the experiment with only 60 % battery charge to better observe the impact of different configurations.

Removing the battery level and temperature checks has significant impact on those values, as shown in Figure 9. Although the battery levels are mostly influenced by satellite power, recovering from the low battery charge takes 10 % longer without state checking, as the system immediately starts processing frames rather than waiting for available power. This is also reflected in chip temperature, which grows beyond the 50 °C limit. On this small experiment

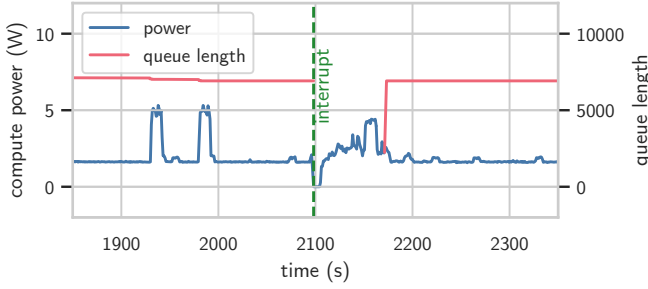


Figure 10: Trabant seamlessly resumes processing after a hard reset, e.g., after an interrupt through SEU

timescale, adding the segment function mainly impacts queue size, with little impact on chip temperature.

5.5 SEU Resilience

Using COTS hardware for satellite computing increases the risk of single-event upsets (SEU) locking up the compute equipment during operation, requiring a hard reset. In Trabant, the FaaS paradigm makes SEU resilience particularly straightforward: as the functions itself are stateless, we simply persist the input invocation queue to find out which software has processed which frames. To evaluate SEU resilience of our Trabant prototype, we run a subset of our experiment and unplug the Raspberry Pi 4 power cable a few minutes into the experiment, essentially hard resetting the device without signalling the operating system. We then plug the cable back in to resume operation.

The compute power draw and queue length observations shown in Figure 10 show how Trabant dequeues some function invocations shortly before our interrupt (normal operation), with power draw increasing accordingly. When we cause the interrupt at 2100 s, power draw drops to 0 W. As soon as we plug the power cable back in, the device boots and resumes its invocation queue.

5.6 Deployment Size

In terms of communication, Earth observation satellites are optimized for downlinking data rather than uplinking new commands, which occurs infrequently. BUPT-1 has a 1 Mbps uplink, which leads to a theoretical maximum upload size of 75 MB per day, assuming a perfect 10 min contact window each day. In reality, this uplink requires error correction and is shared with critical data such as satellite commands. For a shared satellite platform, deployment sizes for tenant software are thus limited.

Figure 11 shows the deployment size of the moisture function in Trabant and as a traditional container image. The model used in the moisture function is the largest of

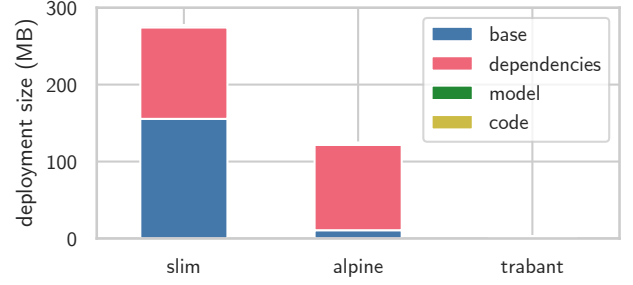


Figure 11: Deployment sizes for the moisture model and code as Docker containers with ‘slim’ and Alpine Linux base images and in Trabant

all of our models at 2.77 MB, but this size is dwarfed by the size of the required base images and dependencies. The official “slim” Python3 base image from Docker Hub has a size of 155.47 MB, while dependencies for model inference are an additional 119 MB. A custom image based on Alpine Linux (which requires manually compiling dependencies such as TensorFlow Lite) has a total size of 121.57 MB. After compression, the slim and Alpine images are 100.47 MB and 39.42 MB large, respectively.

Trabant also requires a base runtime to exist on the satellite, yet runtime files are identical across tenant functions, allowing us to reduce deployment sizes considerably. Despite this, Trabant also allows adding custom dependencies, such as a specific Python library, to deployment packages. Of course, a similar effect could be achieved by coordinating a shared base image among tenants, which would only need to be uploaded once, yet this would increase complexity for service developers.

6 Discussion & Future Work

Our evaluation of the Trabant approach has shown that a serverless architecture is well-suited for shared OEC satellites. We now discuss limitations of our work and derive avenues for future work.

Earth Observation Satellite Constraints. Our evaluation of Trabant is based on traces of the BUPT-1 satellite. We chose this evaluation scenario as it can be realistically replicated in a lab based on COTS hardware and available finely-grained, comprehensive traces of satellite trajectory and power consumption. Of course, BUPT-1 is primarily a research satellite and can thus not compete with state-of-the-art deployed LEO satellites in terms of Earth observation performance. For example, Planet’s High Speed Downlink 2 [28] achieves a 16.25× higher downlink rate than we assume for BUPT-1 (1.3 Gbps compared to 80 Mbps) while performing more frequent passes over ground stations further North than

Tongchuan, China. Similarly, Planet’s *SkySat* constellation promises a ground resolution on the order of 0.5 m/px [4], a 20× higher resolution than that of the Sentinel-2 data used in our evaluation. Finally, more efficient solar arrays, e.g., 135 W [9], are available and more powerful compute resources, e.g., multicore 2nd Generation Intel Xeon Scalable processors with multiple gigabytes of memory on the HPE *Spaceborne Computer-2* [10, 72], have been deployed to LEO. Yet, we note that increasing such parameters does not alleviate the pressure to efficiently downlink data to Earth. With a tenfold increase in compute performance and downlink bandwidth while resolution and service resource requirements also increases tenfold, the entire satellite system would still require the same efficient scheduling as in our evaluation. With Trabant we make no assumptions on the absolute resource constraints, instead we target the relative mismatch between available resources and the desire to run as much processing as possible on as data of the highest possible resolution.

Constellations. Commercial Earth observation satellite operators usually operate tens or hundreds of satellites rather than just one. Constellations of similar satellites allow more frequent frame captures and higher downlink rates, and we imagine that the Trabant approach could also be extended to such constellations. Specifically, given a set of tenant services and a pool of satellites to host these services, we could use the constraints for each satellite set out in §4 to calculate service assignments. A service could also be assigned to multiple satellites in different orbits to yield higher visiting frequency. While we focus our evaluation on just a single satellite in this work, we plan to explore the challenges of such assignments in future work.

Trust Model. Running a service on shared infrastructure raises questions of trust. Specifically, as is also the case in cloud computing, tenants must trust the operator who runs their services. Tenants that process sensitive or otherwise critical data may thus not be able to benefit from the shared satellite approach. We believe that research on trusted serverless computing [20, 68, 84] can be directly applied here. Conversely, the operator can largely treat the tenant service as an untrusted workload given the sandboxing in our serverless environment. Functions cannot access data outside their sandbox and can be monitored if they behave abnormally, e.g., consuming too many CPU cycles, as is standard practice in cloud serverless computing [12].

Service Isolation. Similarly to trust between operator and tenant, we must also consider isolation of services between tenants, a pressing research challenge in serverless computing. Our Trabant prototype uses Docker containers for service management and isolation, yet we are aware that

this is associated with security risks [12, 51]. Alternative sandboxing mechanisms from research on serverless edge computing, such as microVMs [12] or unikernels [35, 53], may provide higher levels of isolation at the cost of efficiency and should be explored for our use-case in future work.

Resource Pricing. We consider the pricing of functions on a Trabant satellite out of scope for this paper, but find it an interesting avenue for future work. As a starting point, it may be useful to adopt pricing models of cloud serverless compute platforms, which charge for processing time and egress bandwidth [30] (with prices adjusted to capital and operational expense of hosting services on a LEO satellite, of course). We believe that this may also encourage efficient service implementations.

Base Resource Consumption. In our experiments, we found that the base power consumption of the Raspberry Pi 4 (1518 mW) consumed over half of the available 2950 mW generated power, a result of the Raspberry Pi not supporting hibernation. Given our approach of time-shifted serverless computing in Trabant, it might instead be possible to power off the device during periods of low power supply and resume operation later, which we will explore in future research.

7 Related Work

Orbital edge computing has drawn considerable research interest [78, 83]. For example, Giuffrida et al. [36, 37] have demonstrated satellite on-board cloud segment using a CNN model running on an Intel Movidius Myriad 2 hardware accelerator on the Φ -Sat-1 satellite. Further, Denby and Lucia [27] introduce an OEC architecture for Earth observation missions that parallelizes data collection and processing across a constellation of nanosatellites, which they evaluate with a CNN model identifying building footprint. With *Kodan*, Denby et al. [26] present an approach for ML inference in space using a selection of pre-trained models optimized to operate under the constraints of a target satellite, with the runtime dynamically selecting the best model based on geospatial contexts. Furutanpey et al. [33] present *FOOL*, an OEC-native feature compression method that leverages inter-tile dependencies to reduce the downlink data size of Earth observation frames in a task-agnostic manner. These approaches are excellent examples of the potential of OEC to reduce the downlink data size in LEO satellites. However, they also illustrate that the focus of most existing OEC research is to reduce the downlink rate of indiscriminate frame collection, e.g., by filtering out pixels obscured by clouds, or to support a single application, i.e., designing the satellite mission to perform a single kind of inference task. We believe that shared satellite platforms offer a more flexible way of leveraging OEC but have not been sufficiently studied.

Lei and Saeed [47] lay out their vision for constellation-as-a-service satellites with heterogeneous compute resources to support a broad range of customer applications. Their proposed system uses containers to host services and allocates resources based on geographical position of the satellite. As no implementation of this system exists at the moment, comparability remains limited. As we have shown with Trabant, a serverless approach requires minimal configuration from customers beyond writing function code and eases deployment logic under the myriad of constraints on an OEC-equipped satellite.

O'Donnel et al. [55] extend Amazon Web Services (AWS) edge computing and networking to a LEO satellite in order to seamlessly manage OEC software as if it was running in a traditional cloud. They use AWS IoT Greengrass for lightweight container orchestration and ML inference with Amazon SageMaker, and evaluate their approach with applications such as fire detection and cloud masking. This approach allows for flexible reconfiguration of the satellite mission by updating models or applications, yet does not allow sharing of the satellite between different tenants.

In the context of the BUPT-1 satellite and the larger *Tiansuan* constellation of OEC research satellites [79], Wang et al. [77] present a study of cloud-native satellites. The authors argue that using cloud-native technologies, e.g., containers and Kubernetes, significantly simplifies the development and operation of OEC satellites and software. Their proposed cloud-native satellite architecture allows flexibly allocating a virtualized pool of satellite resources, such as storage, CPU, or hardware accelerators, among different tenant application functions. This closely aligns with the goals of our work, yet we believe that containers are not well-suited for OEC given deployment sizes and inflexibility in resource allocation.

We have proposed the use of serverless computing for satellite computing in previous work [59, 60] in the slightly different context of LEO in-network computing. Here, compute resources are embedded in LEO satellite communication constellations (instead of Earth observation satellites) to serve clients on the ground with low-latency access to compute services. In this context, stateless serverless functions can be seamlessly migrated across interconnected satellites to maintain a static location from the perspective of a ground observer. While related to this work, communications constellations differ in size, use-case, and capacity to the Earth observation satellites we target with Trabant.

8 Conclusion

Orbital edge computing reduces the data transmission needs of Earth observation satellites by processing sensor data on-board, allowing near-real-time insights while minimizing

downlink costs. Current orbital edge computing architectures are inflexible and require intricate mission planning that involves upfront knowledge of the OEC software.

In this paper, we have argued for shared satellite platforms that allow multiple clients to execute custom code on-board the satellite to filter and analyze sensor data. To enable running unknown code from multiple tenants on satellite hardware with varying power and temperature constraints, we leverage the Function-as-a-Service paradigm. We present Trabant, a serverless satellite software platform that reduces operational complexities using automated time-shifted computing. We demonstrate its capabilities with a proof-of-concept prototype and evaluate it using real satellite computing telemetry data. Our findings suggest that Trabant reduces mission planning overheads, offering a scalable and efficient platform Earth observation missions. Future work on Trabant includes investigating deployment optimization and evaluation on real satellite hardware.

Acknowledgments

We thank Ruolin Xing, Mengwei Xu, and Shangguang Wang from Beijing University of Posts and Telecommunications for their continued support in understanding the intricacies of the BUPT-1 satellite.

Partially funded by the Bundesministerium für Bildung und Forschung (BMBF, German Federal Ministry of Education and Research) in the scope of the Software Campus 3.0 (Technische Universität Berlin) program – 01IS23068.

References

- [1] Humans in the Loop 2020. *Semantic segmentation of aerial imagery*. Humans in the Loop. Retrieved October 18, 2024 from <https://www.kaggle.com/datasets/humansintheloop/semantic-segmentation-of-aerial-imagery>
- [2] Cubesat Market 2023. *KRATOS 1U Ready to Fly Cubesat Platform*. Cubesat Market. Retrieved August 28, 2024 from <https://www.cubesat.market/kratos1uplatform>
- [3] European Space Agency 2023. *Overview of Sentinel-2 Mission*. European Space Agency. Retrieved October 18, 2024 from <https://sentiwiki.copernicus.eu/web/s2-mission>
- [4] Planet Labs 2023. *Planet Imagery Product Specifications*. Planet Labs. Retrieved April 9, 2025 from https://assets.planet.com/docs/Planet_Combined_Imagery_Product_Specs_letter_screen.pdf
- [5] Unibap AB 2023. *SpaceCloudOS & SpaceCloudFW*. Unibap AB. Retrieved October 18, 2024 from <https://unibap.com/wp-content/uploads/2023/12/spacecloud-os-fw.pdf>
- [6] Union of Concerned Scientists 2023. *UCS Satellite Database*. Union of Concerned Scientists. Retrieved August 23, 2024 from <https://www.ucsusa.org/resources/satellite-database>
- [7] National Environmental Satellite, Data, and Information Service (NESDIS) 2023. *Visible Infrared Imaging Radiometer Suite (VIIRS)*. National Environmental Satellite, Data, and Information Service (NESDIS). Retrieved October 18, 2024 from <https://www.nesdis.noaa.gov/our-satellites/currently-flying/joint-polar-satellite-system/visible-infrared-imaging-radiometer-suite-viirs>

- [8] OrbitsEdge, Inc. 2024. *Edge Computing & Micro Datacenters in Space*. OrbitsEdge, Inc. Retrieved October 18, 2024 from <https://orbitsedge.com/edge-in-space>
- [9] Pumpkin Space Systems 2025. *135W Dual Articulated Deployable Solar Array*. Pumpkin Space Systems. Retrieved April 9, 2025 from https://www.pumpkinspace.com/store/p215/135W_Dual_Articulated_Deployable_Solar_Array.html
- [10] NASA Spinoff Technology Transfer Program 2025. *Cutting-Edge Computing Goes Spaceborne*. NASA Spinoff Technology Transfer Program. Retrieved April 9, 2025 from https://spinoff.nasa.gov/Cutting-Edge_Computing_Goes_Spaceborne
- [11] Abdelghani Aaba. 2021. *Wildfire Prediction Dataset (Satellite Images)*. Retrieved October 18, 2024 from <https://www.kaggle.com/datasets/abdelghaniaaba/wildfire-prediction-dataset>
- [12] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight virtualization for serverless applications. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (Santa Clara, CA, USA) (NSDI '20)*. USENIX Association, Berkeley, CA, USA, 419–434.
- [13] Antonio Pertusa Antonio-Javier Gallego and Pablo Gil. 2018. Automatic Ship Classification from Optical Aerial Images with Convolutional Neural Networks. *Remote Sensing* 10, 4, Article 511 (April 2018), 20 pages. doi:10.3390/rs10040511
- [14] Austin P. Arechiga, Alan J. Michaels, and Jonathan T. Black. 2018. Onboard Image Processing for Small Satellites. In *Proceedings of the IEEE National Aerospace and Electronics Conference (Dayton, OH, USA) (NAECON 2018)*. IEEE, New York, NY, USA, 234–240. doi:10.1109/NAECON.2018.8556744
- [15] Mohammad S. Aslanpour, Adel N. Toosi, Claudio Cicconetti, Bahman Javadi, Peter Sbarski, Davide Taibi, Marcos Assuncao, Sukhpal Singh Gill, Raj Gaire, and Schahram Dustdar. 2021. Serverless Edge Computing: Vision and Challenges. In *Proceedings of the 2021 Australasian Computer Science Week Multiconference (Dunedin, New Zealand) (ACSW '21)*. Association for Computing Machinery, New York, NY, USA, 1–10. doi:10.1145/3437378.3444367
- [16] Matej Batič. 2018. *Sentinel Hub Cloud Detector — s2cloudless*. Sentinel Hub. Retrieved October 18, 2024 from <https://medium.com/sentinel-hub/sentinel-hub-cloud-detector-s2cloudless-a67d263d3025>
- [17] Aaron C. Boley and Michael Byers. 2021. Satellite mega-constellations create risks in Low Earth Orbit, the atmosphere and on Earth. *Scientific Reports* 11, Article 10642 (May 2021), 8 pages. doi:10.1038/s41598-021-89909-7
- [18] Jasper Bouwmeester and Jian Guo. 2010. Survey of worldwide pico- and nanosatellite missions, distributions and subsystem technology. *Acta Astronautica* 67, 7–8 (June 2010), 854–862. doi:10.1016/j.actaastro.2010.06.004
- [19] Justin D. Braaten, Warren B. Cohen, and Zhiqiang Yang. 2015. Automated cloud and cloud shadow identification in Landsat MSS imagery for temperate ecosystems. *Remote Sensing of Environment* 169 (Aug. 2015), 128–138. doi:10.1016/j.rse.2015.08.006
- [20] Stefan Brenner and Rüdiger Kapitza. 2019. Trust more, serverless. In *Proceedings of the 12th ACM International Conference on Systems and Storage (Haifa, Israel) (SYSTOR '19)*. Association for Computing Machinery, New York, NY, USA, 33–43. doi:10.1145/3319647.3325825
- [21] Changyong Cao, Xiaoxiong (Jack) Xiong, Robert Wolfe, Frank DeLucia, Quanhua (Mark) Liu, Slawomir Blonski, Guoqing (Gary) Lin, Masahiro Nishihama, Dave Pogorzala, Hassan Oudrari, and Don Hillger. 2013. *Visible Infrared Imaging Radiometer Suite (VIIRS) Sensor Data Record (SDR) User's Guide*. Technical Report 142. National Environmental Satellite, Data, and Information Service, National Oceanic and Atmospheric Administration, U.S. DEPARTMENT OF COMMERCE, Washington, D.C., USA. <https://nsidc.org/sites/default/files/viirs-sdr-users-guide.pdf>
- [22] Jeroen Cappaert. 2018. Building, Deploying and Operating a Cubesat Constellation - Exploring the Less Obvious Reasons Space is Hard. In *Proceedings of the 32nd Annual Small Satellite Conference (Logan, UT, USA) (SmallSat '18)*. Utah State University, Logan, UT, USA.
- [23] Justin Carnahan, Amy Hutputanasin, Alicia Johnstone, Wenschel Lan, Simon Lee, Arash Mehrpavar, Riki Munakata, David Pignatelli, and Armen Toorian. 2022. *Cubesat Design Specification (1U – 12U), Rev. 14.1*. Technical Report CP-CDS-R14.1. California Polytechnic State University, San Luis Obispo, CA, USA. https://www.cubesat.org/s/CDS-REV14_1-2022-02-09.pdf
- [24] Dakai Chen, Edward Wilcox, Raymond L. Ladbury, Christina Seidleck, Hak Kim, Anthony Phan, and Kenneth A. LaBel. 2017. Heavy Ion and Proton-Induced Single Event Upset Characteristics of a 3-D NAND Flash Memory. *IEEE Transactions on Nuclear Science* 65, 1 (Oct. 2017), 19–26. doi:10.1109/TNS.2017.2764852
- [25] Alfredos-Panagiotis Damkalis, Daniel Esparon, and Jack Philpott. 2024. *SatNOGS DB – Kanyini*. Libre Space Foundation. Retrieved August 29, 2024 from <https://db.satnogs.org/satellite/98890#data>
- [26] Bradley Denby, Krishna Chintalapudi, Ranveer Chandra, Brandon Lucia, and Shadi Noghabi. 2023. Kodan: Addressing the Computational Bottleneck in Space. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (Vancouver, BC, Canada) (ASPLOS '23)*. Association for Computing Machinery, New York, NY, USA, 392–403. doi:10.1145/3582016.3582043
- [27] Bradley Denby and Brandon Lucia. 2020. Orbital Edge Computing: Nanosatellite Constellations as a New Class of Computer System. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (Lausanne, Switzerland) (ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 939–954. doi:10.1145/3373376.3378473
- [28] Kiruthika Devaraj, Matt Ligon, Eric Blossom, Joseph Breu, Bryan Klofas, Kyle Colton, and Ryan Kingsbury. 2019. Planet high speed radio: Crossing Gbps from a 3U cubesat. In *Proceedings of the 33rd Annual Small Satellite Conference (Logan, UT, USA) (SmallSat '19)*. Utah State University, Logan, UT, USA.
- [29] Matthias Drusch, Umberto Del Bello, Sébastien Carlier, Olivier Colin, Veronica Fernandez, Ferran Gascon, Bianca Hoersch, Claudia Isola, Paolo Laberinti, Philippe Martimort, et al. 2012. Sentinel-2: ESA's optical high-resolution mission for GMES operational services. *Remote sensing of Environment* 120 (Feb. 2012), 25–36. doi:10.1016/j.rse.2011.11.026
- [30] Tarek Elgamal, Atul Sandur, Klara Nahrstedt, and Gul Agha. 2018. Costless: Optimizing Cost of Serverless Computing through Function Fusion and Placement. In *Proceedings of the 2018 IEEE/ACM Symposium on Edge Computing (Seattle, WA, USA) (SEC '18)*. IEEE, New York, NY, USA, 300–312. doi:10.1109/SEC.2018.00029
- [31] José P. Ferreira, Ziyu Huang, Ken-ichi Nomura, and Joseph Wang. 2024. Potential Ozone Depletion From Satellite Demise During Atmospheric Reentry in the Era of Mega-Constellations. *Geophysical Research Letters* 51, 11, Article e2024GL109280 (June 2024). doi:10.1029/2024GL109280
- [32] Gianluca Furano, Gabriele Meoni, Aubrey Dunne, David Moloney, Veronique Ferlet-Cavrois, Antonis Tavoularis, Jonathan Byrne, Léonie Buckley, Mihalis Psarakis, Kay-Obbe Voss, and Luca Fanucci. 2020. Towards the Use of Artificial Intelligence on the Edge in Space Systems: Challenges and Opportunities. *IEEE Aerospace and Electronic Systems Magazine* 35, 12 (Dec. 2020), 44–56. doi:10.1109/MAES.2020.3008468
- [33] Alireza Furutanpey, Qiyang Zhang, Philipp Raith, Tobias Pfandzelter, Shangguang Wang, and Schahram Dustdar. 2025. FOOL: Addressing

- the Downlink Bottleneck in Satellite Computing with Neural Feature Compression. *IEEE Transactions on Mobile Computing* (Feb. 2025). doi:10.1109/TMC.2025.3544516
- [34] Jose L. Garcia. 2021. 9 - Electric power systems. In *Cubesat Handbook*, Chantal Cappelletti, Simone Battistini, and Benjamin K. Malphrus (Eds.). Academic Press, 185–197. doi:10.1016/B978-0-12-817884-3.00009-6
- [35] Dániel Géhberger and Dávid Kovács. 2022. Cooling down faas: Towards getting rid of warm starts. (June 2022). arXiv:2206.00599
- [36] Gianluca Giuffrida, Lorenzo Diana, Francesco de Gioia, Gionata Benelli, Gabriele Meoni, Massimiliano Donati, and Luca Fanucci. 2020. Cloud-Scout: A deep neural network for on-board cloud detection on hyperspectral images. *Remote Sensing* 12, 14, Article 2205 (July 2020), 17 pages. doi:10.3390/rs12142205
- [37] Gianluca Giuffrida, Luca Fanucci, Gabriele Meoni, Matej Batič, Léonie Buckley, Aubrey Dunne, Chris Van Dijk, Marco Esposito, John Hefe, Gianluca Vercruyssen, Nathan Furano, Massimiliano Pastena, and Josef Aschbacher. 2021. The Φ -Sat-1 Mission: The First On-Board Deep Neural Network Demonstrator for Satellite Earth Observation. *IEEE Transactions on Geoscience and Remote Sensing* 60 (Nov. 2021), 1–14. doi:10.1109/TGRS.2021.3125567
- [38] Jared Michael Greene, Mohammed Faraz Admani, Jacob Nelson Glueck, Sergii Ziuzin, Francesco De Paolis, Dhruv Dawar, and Christopher Yu. 2022. System and method of providing access to compute resources distributed across a group of satellites. US Patent App. US20230164089A1: filed Sep. 28., 2022.
- [39] Ehab H. Hegazi, Abdellateif A. Samak, Lingbo Yang, Ran Huang, and Jingfeng Huang. 2023. Prediction of Soil Moisture Content from Sentinel-2 Images Using Convolutional Neural Network (CNN). *Agronomy* 13, 3, Article 656 (Feb. 2023), 18 pages. doi:10.3390/rs10040511
- [40] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. 2019. EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 12, 7 (June 2019), 2217–2226. doi:10.1109/JSTARS.2019.2918242
- [41] Scott Hendrickson, Stephen Sturdevant, Tyler Harter, Venkateshwaran Venkataramani, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2016. Serverless Computation with OpenLambda. In *Proceedings of the 8th USENIX Workshop on Hot Topics in Cloud Computing* (Denver, CO, USA) (*HotCloud '16*). USENIX Association, Berkeley, CA, USA, 33–39.
- [42] Maximilian Henkel, Vladimir Zelenevskiy, Georges Labrèche, Rodrigo Laurinovic, David Evans, Dominik Marszk, and Omiros Papadatos Vasilakis. 2024. Mitigating and Recovering from Radiation Induced Faults in Non-Hardened Spacecraft Flash Memory. In *Proceedings of the 2024 IEEE Aerospace Conference* (Big Sky, MT, USA) (*AERO '24*). IEEE, New York, NY, USA, 1–8. doi:10.1109/AERO58975.2024.10521125
- [43] Brent Horine. 2021. Creating a Marketplace for a Constellation as a Service. In *Proceedings of the 38th Annual Small Satellite Conference* (Logan, UT, USA) (*SmallSat '21*). Utah State University, Logan, UT, USA.
- [44] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Menezes Carreira, Karl Krauth, Neeraja Yadwadkar, Joseph Gonzalez, Raluca Ada Popa, Ion Stoica, and David A. Patterson. 2019. *Cloud Programming Simplified: A Berkeley View on Serverless Computing*. Technical Report UCB/EECS-2019-3. EECS Department, University of California, Berkeley, Berkeley, CA, USA. <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2019/EECS-2019-3.html>
- [45] Donald J. Kessler and Burton G. Cour-Palais. 1978. Collision frequency of artificial satellites: The creation of a debris belt. *Journal of Geophysical Research: Space Physics* 83, 6 (June 1978), 2637–2646. doi:10.1029/JA083iA06p02637
- [46] Michael D. King, Steven Platnick, W. Paul Menzel, Steven A. Ackerman, and Paul A. Hubanks. 2013. Spatial and Temporal Distribution of Clouds Observed by MODIS Onboard the Terra and Aqua Satellites. *IEEE Transactions on Geoscience and Remote Sensing* 51, 7 (Jan. 2013), 3826–3852. doi:10.1109/TGRS.2012.2227333
- [47] Demi Lei and Ahmed Saeed. 2024. Do We Need a Million Satellites in Orbit? Constellation-as-a-Service with Modular Satellites: Challenges and Opportunities. In *Proceedings of the 2nd International Workshop on LEO Networking and Communication* (Washington, DC, USA) (*LEO-NET '24*). Association for Computing Machinery, New York, NY, USA, 61–66. doi:10.1145/3697253.3697262
- [48] Israel Leyva-Mayorga, Marc Martinez-Gost, Marco Moretti, Ana Pérez-Neira, Miguel Ángel Vázquez, Petar Popovski, and Beatriz Soret. 2023. Satellite Edge Computing for Real-Time and Very-High Resolution Earth Observation. *IEEE Transactions on Communications* 71, 10 (July 2023), 6180–6194. doi:10.1109/TCOMM.2023.3296584
- [49] Sha Lu, Eriita Jones, Liang Zhao, Yu Sun, Kai Qin, Jixue Liu, Jiuyong Li, Prabath Abeysekara, Norman Mueller, Simon Oliver, Jim O’Hehir, and Stefan Peters. 2024. Onboard AI for Fire Smoke Detection using Hyperspectral Imagery: an Emulation for the Upcoming Kanyini Hyperscout-2 Mission. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 17 (April 2024), 9629–9640. doi:10.1109/JSTARS.2024.3394574
- [50] Catherine G. Manning and Abigail Bowman. 2023. *SCaN Glossary*. National Aeronautics and Space Administration, Space Communications and Navigation (SCaN) Program. Retrieved August 28, 2024 from <https://www.nasa.gov/reference/scan-glossary/>
- [51] Eyal Manor. 2018. *Bringing the best of serverless to you*. Google Cloud Platform. Retrieved February 12, 2024 from <https://cloudplatform.googleblog.com/2018/07/bringing-the-best-of-serverless-to-you.html>
- [52] Jonathan C. McDowell. 2020. The Low Earth Orbit Satellite Population and Impacts of the SpaceX Starlink Constellation. *The Astrophysical Journal Letters* 892, 2, Article L36 (April 2020), 10 pages. doi:10.3847/2041-8213/ab8016
- [53] Felix Moebius, Tobias Pfandzelter, and David Bermbach. 2024. Are Unikernels Ready for Serverless on the Edge?. In *Proceedings of the 12th IEEE International Conference on Cloud Engineering* (Paphos, Cyprus) (*IC2E '24*). IEEE, New York, NY, USA, 133–143. doi:10.1109/IC2E61754.2024.00022
- [54] Cristóbal Nieto-Peroy and M. Reza Emami. 2019. CubeSat Mission: From Design to Operation. *Applied Sciences* 9, 15, Article 3110 (Aug. 2019), 24 pages. doi:10.3390/app9153110
- [55] Kathryn O’Donnell, Meghan Weber, Joy Fasnacht, Jeff Maynard, Margaret Cote, and Shayn Hawthorne. 2023. Extension of cloud computing to small satellites. In *Proceedings of the 37th Annual Small Satellite Conference* (Logan, UT, USA) (*SmallSat '23*). Utah State University, Logan, UT, USA, 304–318.
- [56] Nathan Pemberton and Johann Schleier-Smith. 2019. The serverless data center: Hardware disaggregation meets serverless computing. In *Proceedings of the The First Workshop on Resource Disaggregation* (Providence, RI, USA) (*WORD '19*). http://word19.ece.cornell.edu/serverless_data_center.pdf
- [57] Tobias Pfandzelter and David Bermbach. 2020. tinyFaaS: A Lightweight FaaS Platform for Edge Environments. In *Proceedings of the Second IEEE International Conference on Fog Computing* (Sydney, NSW, Australia) (*ICFC 2020*). IEEE, New York, NY, USA, 17–24. doi:10.1109/ICFC49376.2020.00011
- [58] Tobias Pfandzelter and David Bermbach. 2023. Edge Computing in Low-Earth Orbit – What Could Possibly Go Wrong?. In *Proceedings of the the 1st ACM Workshop on LEO Networking and Communication 2023*

- (Madrid, Spain) (*LEO-NET '23*). Association for Computing Machinery, New York, NY, USA, 19–24. doi:10.1145/3614204.3616106
- [59] Tobias Pfandzelter and David Bermbach. 2024. Komet: A Serverless Platform for Low-Earth Orbit Edge Services. In *Proceedings of the 15th ACM Symposium on Cloud Computing* (Redmond, WA, USA) (SoCC '24). Association for Computing Machinery, New York, NY, USA, 866–882. doi:10.1145/3698038.3698517
- [60] Tobias Pfandzelter, Jonathan Hasenburger, and David Bermbach. 2021. Towards a Computing Platform for the LEO Edge. In *Proceedings of the 4th International Workshop on Edge Systems, Analytics and Networking* (Online, United Kingdom) (*EdgeSys '21*). Association for Computing Machinery, New York, NY, USA, 43–48. doi:10.1145/3434770.3459736
- [61] Ryne P. Raffaele. 2023. 9 - Introduction to CubeSat power systems. In *Next Generation CubeSats and SmallSats*, Francesco Branz, Chantal Cappelletti, Antonio J. Ricco, and John W. Hines (Eds.). Elsevier, 201–221. doi:10.1016/B978-0-12-824541-5.00008-X
- [62] Philipp Raith, Stefan Nastic, and Schahram Dustdar. 2023. Serverless Edge Computing – Where We Are and What Lies Ahead. *IEEE Internet Computing* 27, 3 (May 2023), 50–64. doi:10.1109/MIC.2023.3260939
- [63] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation. In *Proceedings of the Medical image computing and computer-assisted intervention* (Munich, Germany) (*MICCAI '15*). Springer, Cham, Switzerland, 234–241. doi:10.1007/978-3-319-24574-4_28
- [64] Robert G. Ryan, Eloise A. Marais, Chloe J. Balhatchet, and Sebastian D. Eastham. 2022. Impact of Rocket Launch and Space Debris Air Pollutant Emissions on Stratospheric Ozone and Global Climate. *Earth's Future* 10, 6, Article e2021EF002612 (June 2022). doi:10.1029/2021EF002612
- [65] Alireza Sahraei, Soteris Demetriou, Amirali Sobhghol, Haoran Zhang, Abhigna Nagaraja, Neeraj Pathak, Girish Joshi, Carla Souza, Bo Huang, Wyatt Cook, Andrii Golovei, Pradeep Venkat, Andrew Mcfague, Dimitrios Skarlatos, Vipul Patel, Ravinder Thind, Ernesto Gonzalez, Yun Jin, and Chunqiang Tang. 2023. XFaaS: Hyperscale and Low Cost Serverless Functions at Meta. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) (*SOSP '23*). Association for Computing Machinery, New York, NY, USA, 231–246. doi:10.1145/3600006.3613155
- [66] Antonio Pertusa Samer Alashhab, Antonio-Javier Gallego and Pablo Gil. 2019. Precise Ship Location With CNN Filter Selection From Optical Aerial Images. *IEEE Access* 7 (July 2019), 96567–96582. doi:10.1109/ACCESS.2019.2929080
- [67] Trever Schirmer, Valentin Carl, Tobias Pfandzelter, and David Bermbach. 2023. ProFaaSinate: Delaying Serverless Function Calls to Optimize Platform Performance. In *Proceedings of the 9th International Workshop on Serverless Computing* (Bologna, Italy) (*WoSC '23*). Association for Computing Machinery, New York, NY, USA, 1–6. doi:10.1145/3631295.3631393
- [68] Avishka Shamendra, Binoy Peries, Gayangi Seneviratne, and Sunimal Rathnayake. 2023. TruFaaS-Trust Verification Framework for FaaS. In *Proceedings of the International Conference on Ubiquitous Security* (Exeter, UK) (*UbiSec '23*). Springer, Cham, Switzerland, 304–318. doi:10.1007/978-981-97-1274-8_20
- [69] Weisong Shi and Schahram Dustdar. 2016. The Promise of Edge Computing. *Computer* 49, 5 (May 2016), 78–81. doi:10.1109/MC.2016.145
- [70] Windy S. Slater, Nayana P. Tiwari, Tyler M. Lovelly, and Jesse K. Mee. 2020. Total Ionizing Dose Radiation Testing of NVIDIA Jetson Nano GPUs. In *Proceedings of the 2020 IEEE High Performance Extreme Computing Conference* (Waltham, MA, USA) (*HPEC '20*). IEEE, New York, NY, USA, 1–3. doi:10.1109/HPEC43674.2020.9286222
- [71] Beatriz Soret, Israel Leyva-Mayorga, Antonio M. Mercado-Martínez, Marco Moretti, Antonio Jurado-Navas, Marc Martínez-Gost, Celia Sánchez de Miguel, Ainoa Salas-Prendes, and Petar Popovski. 2024. Semantic and goal-oriented edge computing for satellite Earth Observation. (Aug. 2024). arXiv:2408.15639
- [72] Richard Speed. 2021. *HPE Spaceborne Computer-2 slips off the shelf – and off the planet: Boxen heading to ISS*. The Register. Retrieved April 9, 2025 from https://www.theregister.com/2021/02/11/hpe_spaceborne_2_iss/
- [73] Gencer Sumbul, Marcela Charfuelan, Begüm Demir, and Volker Markl. 2019. Bigearthnet: A Large-Scale Benchmark Archive for Remote Sensing Image Understanding. In *Proceedings of the 2019 IEEE International Geoscience and Remote Sensing Symposium* (Yokohama, Japan) (*IGARSS '19*). IEEE, New York, NY, USA, 5901–5904. doi:10.1109/IGARSS.2019.8900532
- [74] Daniel J Varon, Dylan Jervis, Jason McKeever, Ian Spence, David Gains, and Daniel J Jacob. 2021. High-frequency monitoring of anomalous methane point sources with multispectral Sentinel-2 satellite observations. *Atmospheric Measurement Techniques* 14, 4 (April 2021), 2771–2785. doi:10.5194/amt-14-2771-2021
- [75] Deepak Vasisht, Jayanth Shenoy, and Ranveer Chandra. 2021. L2D2: low latency distributed downlink for LEO satellites. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference* (Virtual Event, USA) (*SIGCOMM '21*). Association for Computing Machinery, New York, NY, USA, 151–164. doi:10.1145/3452296.3472932
- [76] Danton José Fortes Villas Boas, José Bezerra Pessoa Filho, Alison de Oliveira Moraes, and Carlos Henrique Melo Souza. 2023. 17 - Innovative and low-cost launch systems. In *Next Generation CubeSats and SmallSats*, Francesco Branz, Chantal Cappelletti, Antonio J. Ricco, and John W. Hines (Eds.). Elsevier, 403–419. doi:10.1016/B978-0-12-824541-5.00005-4
- [77] Chao Wang, Yiran Zhang, Qing Li, Ao Zhou, and Shangguang Wang. 2023. Satellite Computing: A Case Study of Cloud-Native Satellites. In *Proceedings of the 2023 IEEE International Conference on Edge Computing and Communications* (Chicago, IL, USA) (*EDGE '23*). IEEE, New York, NY, USA, 262–270. doi:10.1109/EDGE60047.2023.00048
- [78] Shangguang Wang and Qing Li. 2023. Satellite Computing: Vision and Challenges. *IEEE Internet of Things Journal* 10, 24 (Aug. 2023), 22514–22529. doi:10.1109/JIOT.2023.3303346
- [79] Shangguang Wang, Qing Li, Mengwei Xu, Xiao Ma, Ao Zhou, and Qibo Sun. 2021. Tiansuan Constellation: An Open Research Platform. In *Proceedings of the 2021 IEEE International Conference on Edge Computing* (Chicago, IL, USA) (*EDGE '21*). IEEE, New York, NY, USA, 94–101. doi:10.1109/EDGE53862.2021.00022
- [80] Renchao Xie, Qinqin Tang, Shi Qiao, Han Zhu, F. Richard Yu, and Tao Huang. 2021. When Serverless Computing Meets Edge Computing: Architecture, Challenges, and Open Issues. *IEEE Wireless Communications* 28, 5 (July 2021), 126–133. doi:10.1109/MWC.001.2000466
- [81] Ruolin Xing, Mengwei Xu, Ao Zhou, Qing Li, Yiran Zhang, Feng Qian, and Shangguang Wang. 2024. Deciphering the Enigma of Satellite Computing with COTS Devices: Measurement and Analysis. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking* (Washington D.C., DC, USA) (*MobiCom '24*). Association for Computing Machinery, New York, NY, USA, 420–435. doi:10.1145/3636534.3649371
- [82] Boris Yendler. 2021. 16 - Thermal control system. In *Cubesat Handbook*, Chantal Cappelletti, Simone Battistini, and Benjamin K. Malphrus (Eds.). Academic Press, 303–317. doi:10.1016/B978-0-12-817884-3.00016-3
- [83] Yin Zengshan, Wu Changhao, Guo Chongbin, Li Yuanchun, Xu Mengwei, Gao Weiwei, and Chi Chuanxiu. 2024. A comprehensive survey of orbital edge computing: Systems, applications, and algorithms.

Chinese Journal of Aeronautics, Article 3316 (Nov. 2024), 30 pages.
[doi:10.1016/j.cja.2024.11.026](https://doi.org/10.1016/j.cja.2024.11.026) [arXiv:2306.00275](https://arxiv.org/abs/2306.00275)

- [84] Denghui Zhang, Lijing Ren, Muhammad Shafiq, and Zhaoquan Gu. 2022. A Lightweight Privacy-Preserving System for the Security of

Remote Sensing Images on IoT. *Remote Sensing* 14, 24, Article 6371 (Dec. 2022), 16 pages. [doi:10.3390/rs14246371](https://doi.org/10.3390/rs14246371)